

Computer Organization and Architecture Laboratory

MiniRISC Assignment 1A: ISA Planning and Processor Architecture

Indian Institute of Technology Kharagpur
Department of Computer Science and Engineering

Deadline: 5 October 2026

Objective

The objective of this assignment is to define the **complete planned instruction-set architecture** of the 32-bit MiniRISC processor and to prepare the corresponding **processor-level architectural diagram** before beginning the detailed Verilog implementation.

This assignment is primarily a **design and planning exercise**. The ISA prepared here should cover not only the instructions required immediately, but also the instructions that will be required in the later MiniRISC assignments, including multiplication and multiply-accumulate support.

The 32-bit Booth multiplier and MAC datapath are not to be implemented in this assignment. However, the corresponding instruction(s) must be considered during ISA planning so that the processor architecture can be extended cleanly later.

1. Basic Processor Requirements

The planned MiniRISC processor should contain at least:

- sixteen 32-bit general-purpose registers, $R0-R15$;
- a register file with two read ports and one write port;
- $R0$ hard-wired to zero;
- a 32-bit Program Counter (PC);
- a 32-bit ALU;
- instruction decode and control logic;
- immediate-generation logic;
- load/store support;
- branch and program-control support;
- a provision for later integration of the 32-bit fast Booth multiplier;
- instruction and data-memory interfaces.

2. ISA Planning

Each group must design its own MiniRISC instruction encoding.

No fixed opcode map or instruction format is prescribed.

Each group must define:

1. instruction formats;
2. opcode allocation;
3. source- and destination-register fields;
4. immediate fields and extension rules;
5. branch displacement format;
6. memory-addressing format;
7. any special fields required for shift or control instructions.

The ISA should be kept simple, but it must be sufficiently expressive to support the later program assignments.

3. Instruction Capabilities to be Planned

The ISA must include instructions, or equivalent functionality, for the following classes.

3.1 Arithmetic Instructions

Plan instructions equivalent to:

ADD, SUB, ADDI, SUBI.

Additional instructions such as INC and DEC may be included if desired.

3.2 Logical Instructions

Plan sufficient logical operations such as:

AND, OR, XOR, NOR, NOT.

Immediate logical variants may also be included.

3.3 Shift Instructions

The planned ISA should include sufficient shift capability, for example:

SLL/SL, SRL, SRA.

The shift amount may be supplied by a register or by an immediate field.

3.4 Comparison Instructions

Plan at least one mechanism for comparison, for example:

SLT, SGT,

or another equivalent approach.

3.5 Memory Instructions

Plan instructions equivalent to:

LD and ST.

A base-register plus immediate-offset addressing mode is recommended because it will be useful for arrays, matrices, and image data.

3.6 Branch and Program-Control Instructions

The planned ISA must support:

- unconditional branch;
- conditional branch based on zero and/or sign;
- NOP;
- HALT.

Instructions equivalent to BZ, BMI, and BPL are convenient, but groups may choose an alternative branch organization.

3.7 Register Transfer and Constant Formation

The ISA should provide a convenient way to:

- copy register values;
- load small immediate values;
- construct larger constants if required.

Instructions such as MOVE or LUI may be used, but are not mandatory if equivalent functionality is provided differently.

3.8 Multiplication / Multiply–Accumulate Planning

The 2026 processor will later be extended using the previously designed **32-bit fast Booth multiplier**.

Therefore, the ISA planning in this assignment must already include either:

$\boxed{\text{MUL } R_d, R_{s1}, R_{s2}}$

or a multiply-accumulate instruction such as

$\boxed{\text{MAC } R_d, R_{s1}, R_{s2}}.$

Groups may also include both.

For the planned multiplication instruction, clearly document:

- source registers;
- destination register;
- whether the operation is signed;
- which part of the 64-bit multiplier result is architecturally visible;
- whether the instruction is expected to be multi-cycle;
- whether processor stalling or software-inserted NOPs may be used later.

Only ISA-level planning is required at this stage. No multiplier or MAC hardware is to be implemented for Assignment 1A.

4. Additional Instructions

Groups may introduce additional instructions if they are useful for later programs.

Possible examples include:

- compare-and-branch;
- load-immediate;
- conditional move;
- loop-control instructions;
- address-generation helper instructions;
- specialized shift or bit-manipulation instructions.

Every additional instruction must be clearly documented and justified.

5. Mandatory Processor Architecture Diagram

Before beginning detailed Verilog coding, every group must prepare a **hand-drawn architectural diagram** and show it to a Teaching Assistant.

The diagram must be at the block level and must clearly show:

1. Program Counter and PC-update path;

2. instruction input / future instruction-memory interface;
3. instruction decoder / control unit;
4. 16×32 register file with two read ports and one write port;
5. immediate-generation logic;
6. ALU input multiplexers;
7. 32-bit ALU;
8. branch-decision / PC-selection path;
9. data-memory interface;
10. write-back path;
11. a clearly marked **future multiplier / MAC integration point**;
12. major data buses and widths;
13. major control signals.

The diagram must clearly distinguish:

Datapath and Control Path.

The multiplier / MAC block may be shown only as a **future placeholder**; it is not implemented in this assignment.

6. Required Submission

By the deadline, each group must submit:

- the hand-drawn or cleanly redrawn processor architecture;
- the complete planned instruction list;
- proposed instruction formats;
- proposed opcode allocation;
- a short explanation of immediate, branch, and memory-addressing conventions;
- the planned MUL and/or MAC instruction semantics;
- a short justification of the chosen ISA organization.

No complete processor Verilog implementation is required for this 5 October submission.